

Mario RL: What Works, What We Did, and Why

Double-DQN agent for Super Mario Bros 1-1, built and training on a CPU-only Windows + WSL machine. Ends with design options for a Pokemon agent. Snapshot taken September 14, 2026, ~20 minutes into the full run.

Status at a glance

Component	State	Evidence
Environment (WSL, Python 3.10, pinned deps)	Working	nes-py compiled with gcc; import probe passes; requirements.txt frozen
Env adapter (env/make_env.py)	Working	4 env tests pass against the real emulator
Frame replay buffer (agent/buffer.py)	Working	3 tests: every sample matches naive storage, incl. wraparound and 1-step episodes
Double-DQN agent + checkpoints (agent/mario.py)	Working	6 tests: resumed agent makes the identical next update and actions
train.py / eval.py / config presets	Working	Smoke run, resume with buffer, resume without buffer, eval all run end to end
Detached launcher + clean stop	Working	SIGTERM test wrote a final checkpoint; full run survives closing the terminal
Full training run dqn_1_1	In progress	Step 136,735 of 3,000,000; epsilon 0.877; ~120 steps/s; ETA about 11:30 PM to midnight
Trained agent quality	Unknown yet	Needs eval.py once epsilon reaches 0.1 at step 1M

Test suite: 14 of 14 passing. Git: two local commits on **main**, nothing pushed.

1. Reviewed the plan before building

The starting point was a seven-phase plan to fix the PyTorch Mario tutorial. Most of it was sound. Before writing code, each claim was checked against the gym-super-mario-bros source and, later, the running emulator. Four reasons were wrong even where the fix was right, and four things were missing.

Plan claim	What is actually true	Outcome
Mario's 400-tick clock causes truncation, so bootstrapping is broken (the top-priority fix).	Timer expiry kills Mario : the probe ran NOOPs until time=0 and got terminated=True, truncated=False. gym's TimeLimit is registered at 9,999,999 steps, so truncated is never set.	Kept the terminated/truncated split (cheap and correct) but demoted it. It only matters with our own cutoff.
Hardcoded batch_size in Q[np.arange(B), action] silently indexes wrong rows.	Mismatched lengths usually raise . The silent bug is an action shaped [B,1], which broadcasts to a BxB matrix.	Kept gather(), plus shape asserts and squeeze(-1) everywhere.
NumPy 2 breaks nes-py's C extension and segfaults.	The confirmed break is in Python: smb_env computes uint8 x 256 for x_pos, which NumPy 2 rejects.	Kept the numpy<2 pin with the correct reason.
Restarting without Adam state wrecks a model.	Bias correction makes it a jolt, not a wreck. The bigger resume hole was the empty replay buffer .	Checkpoints save optimizer + RNG; learning waits for burn-in when the buffer is not restored.

Missing from the plan, added:

- **Reward scaling.** Per-frame reward is clipped to +/-15; four skipped frames sum to +/-60. With Huber loss (beta=1) errors that large sit on the linear part, so every gradient has the same magnitude. Stored reward is divided by 15; logs and eval stay in raw units.
- **No-progress cutoff.** A Mario stuck on a pipe burns about 8,000 frames of useless data. Episodes are truncated if the best x_pos has not improved in 150 agent steps. This is a genuine truncation, where bootstrapping is correct.
- **Eval that detects memorization.** The NES emulator is deterministic, so a near-greedy policy replays the same run. Eval uses per-episode seeds, random NOOP starts, held-out stages, and counts unique action sequences.
- **A smoke preset that exercises everything.** The tutorial syncs the target every 10k steps and saves every 500k, so a short test never reaches those paths. Smoke syncs every 500 and saves every 1,000.

2. Set up the environment

Finding	Decision	Why
GPU is an AMD RX 5700 XT	Train on CPU (Ryzen 7 7800X3D, 16 threads)	PyTorch CUDA needs NVIDIA; ROCm does not support this card on Windows. The emulator is the bottleneck for a small CNN anyway.
nes-py ships only a source tarball for Windows; VS Build Tools had no C++ compiler	Run everything in WSL Ubuntu 24.04	gcc 13 was already installed; no admin rights or 6-8 GB download needed.
nes-py targets the old gym API and NumPy 1.x	Python 3.10, gym 0.26.2, numpy 1.26.4, torch 2.14 CPU	Locking the interpreter is cheaper than patching a C extension we do not own. Frozen right after the import test.
Linux venv on /mnt/c is slow	Venv in ~/.venvs/mario-rl; code stays in C:\Users\17143\mario-rl	Code is editable from Windows; Python startup and imports stay fast.

3. Built the project

mario-rl/ config.py env/make_env.py agent/buffer.py agent/net.py agent/mario.py train.py eval.py scripts/ tests/	dataclass presets (smoke, full) + hash of learning fields the ONLY file that knows gym/nes-py quirks frame-deduplicated replay buffer Nature-DQN CNN; uint8 in, /255 inside forward() act / cache / learn / save / load training loop with CSV log; fixed-protocol evaluation probe_env.py, start_train.ps1, stop_train.ps1, wsl_run.sh buffer, agent, env (14 tests)
---	--

Environment adapter

Everything outside **make_env.py** sees a Gymnasium-style API: `reset(seed)` returns (obs, info), `step` returns (obs, reward, terminated, truncated, info), and `obs` is uint8 [4, 84, 84]. Inside, it does frame skip (stopping on either flag), grayscale + INTER_AREA resize, frame stacking, seeded 0-30 NOOP starts, and the no-progress cutoff. Quirks it absorbs: `nes-py's JoypadSpace.reset()` rejects a seed argument, and gym's compatibility layer returns an empty info dict from `reset()`. A test caught that second one: it made the first step always look like progress.

Replay buffer: store each frame once

Storage scheme	Bytes per transition	100k transitions
float32 state + next_state (tutorial)	$4 \times 84 \times 84 \times 4 \text{ bytes} \times 2 = 225,792$	22.6 GB (spills to disk mmap)
uint8 state + next_state (original plan)	$4 \times 84 \times 84 \times 2 = 56,448$	5.6 GB
uint8, one frame per row (built)	$84 \times 84 = 7,056$	706 MB

Consecutive stacks share three of four frames, so the buffer stores only each observation's newest frame and rebuilds stacks at sample time, repeating the first frame across episode starts exactly as `reset()` does. An episode's final observation gets its own row so a truncated transition still has a real next state. This also removed `torchrl` and `tensordict`, which must be version-matched to `torch`.

Agent

- Double DQN: online network picks a' , target network scores it; both lookups use `gather()`.
- TD target = $r + \gamma * (1 - \text{terminated}) * Q_{\text{target}}(s', a')$. Truncation keeps the bootstrap.
- Linear epsilon 1.0 to 0.1 over 1M steps. The tutorial's 0.99999975 multiplier needs $\sim 9.2\text{M}$ steps to reach 0.1.
- γ 0.99 (tutorial 0.9 looks only ~ 10 agent steps, about 0.7 s of game time, ahead). Gradient norm clipped at 10.
- Checkpoint: weights, target, Adam state, step, episode, all RNG states, config, learning hash, and optionally the buffer. Resuming with a changed learning field is refused unless `--allow-config-change` is passed.

4. Verified

Check	Result
Unit + integration tests	14/14 pass (about 6 s)
Smoke training, 2,000 steps	~ 20 s; learning, 3 target syncs, 2 checkpoints; best smoke episode reached <code>x_pos</code> 1510
Resume with saved buffer	Continued at step 2,000 with eps and buffer intact
Resume without buffer	Refilled 200 transitions, then resumed learning
Learning-field override on resume	Refused with a clear message
SIGTERM during training	Caught; final checkpoint written
eval.py on smoke checkpoint	10 episodes, 10 unique trajectories, mean <code>x_pos</code> 625 (expected for 3k steps)

5. Training now

Run **dqn_1_1** started around 4:30 PM with the full preset: 3M steps, 100k buffer, burn-in 10k, checkpoint every 100k steps including the buffer. At step 136,735 the agent is still 88% random, so reward is mostly noise. Mean Q is rising slowly (about 3.3) with small, stable loss.

- **Watch x_pos.** Random play stalls around 700-722, the tall pipes of 1-1. Clearing them consistently is the first sign of real learning.
- **Sleep pauses training.** Keep the PC awake until it finishes.
- **First real measurement:** eval.py on the step-1M checkpoint, on 1-1 and on held-out 1-2.

```
# PowerShell: watch / stop + save / resume
Get-Content C:\Users\17143\mario-rl\runs\dqn_1_1\stdout.log -Tail 5 -Wait
C:\Users\17143\mario-rl\scripts\stop_train.ps1
cd C:\Users\17143\mario-rl; .\scripts\start_train.ps1 -Resume runs/dqn_1_1/latest.pt
```

```
# WSL venv: evaluate on 1-1, then on held-out 1-2
python eval.py runs/dqn_1_1/latest.pt --episodes 30
python eval.py runs/dqn_1_1/latest.pt --stage 2
```

Next: a Pokemon agent

Pokemon demands longer-horizon reasoning than Mario. This section covers what changes, the build options, and where to draw the line on giving the agent outside knowledge.

Why it is harder

	Mario 1-1	Pokemon Red
Reward density	Every step: moving right pays	Badges and new areas arrive thousands of steps apart
Memory needed	4 frames is enough	Where have I been, what am I carrying, which events happened
Episode end	Death or flag (true terminals)	No death; nearly every episode is truncated
Reasoning	Timing jumps	Type matchups, switching, item and HM gating, route planning
Action structure	5 buttons, immediate effect	Menus nested several levels deep

Build options

Option	How	Realistic on this PC
A. Pokemon Red: explore + battle	PyBoy Game Boy emulator (fast, headless, Python). PPO across ~12-16 parallel emulators. Reward read from RAM. Prior art: Peter Whidden's open-source PokemonRedExperiments.	Leaving Pallet Town and beating Brock over a day or two of CPU. The full game is not realistic.
B. Battles only (Showdown)	poke-env Python library against a local Pokemon Showdown server. Structured state (HP, types, moves, boosts); reward is win/loss.	Very doable; fast on CPU. Most type-matchup and switching reasoning per training hour. Needs no ROM.
C. LLM agent plays Red	A language model reads screen + state and chooses actions, like "Claude plays Pokemon".	Easy to run but pays API cost per step, and it is prompting rather than RL.

Recommendation: B first, then A. B isolates the reasoning with a clean reward and needs no ROM. A is the long-horizon exploration problem and reuses B's battle lessons.

The "file that knows where everything is" question

The instinct to avoid it is right, but two different things share that description. One is fine and standard; the other turns the agent into a script.

Level	What the agent gets	Verdict
0	Pixels only; reward only for badges	Fair but hopeless: reward too sparse to ever find
1	RAM addresses used for the reward only: map ID, x/y, party levels, badge bits, event flags. The agent never sees them.	Use. This defines the objective; it does not tell the agent how to reach it. The address table comes from the community poked disassembly.
2	RAM values in the observation: party HP, levels, current map, items	Use. A human player sees all of this on screen or in menus.

Level	What the agent gets	Verdict
3	A visited-tiles map the agent builds itself during the episode	Use. It is the agent's own memory, not outside knowledge.
4	An imported world map, route, or walkthrough telling it where to go next	Do not use. It follows an answer key, learns nothing that transfers, and eval scores measure the file instead of the agent.

Rule of thumb: game memory may tell us *how the agent is doing* (reward) and *what a player could see* (observation), but never *what to do next*. Knowing where a value lives in RAM is a lookup table for the emulator, not memorization of the game.

One trap sits between levels 1 and 4: reward shaping tuned to one route. For example, paying for specific coordinates in Viridian Forest in a specific order quietly encodes the walkthrough. Keep shaped rewards generic: any new tile, any level gained, any badge, any new event flag.

What carries over from mario-rl

Reuse as-is	Change
Single adapter file isolating emulator quirks	DQN becomes PPO with parallel environments (on-policy, no replay buffer)
Config presets with a smoke test that hits every code path	Add recurrent memory (LSTM) or the explicit visited-tiles map
Checkpoints that resume exactly; learning-hash guard	Truncation is the normal case; bootstrap on it correctly
Fixed-seed eval with held-out tests and unique-trajectory count	Eval on save states the agent never trained from, or unseen Showdown teams
Detached launcher, CSV logging, pinned environment	Parallel envs compete with Mario for the same 16 threads

Before starting

- **ROM (option A only):** dump your own cartridge with a USB reader, such as an Epilogue Joey Jr. or insideGadgets GBxCart RW with FlashGBX. Back up the save too. Keep *.gb out of git; we verify the checksum before building on it.
- **CPU:** start after the Mario run finishes, or pause Mario first.
- **Process:** same as Mario. A written plan to review, then build, test, smoke run, full run.